HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.

HỆ GỢI Ý (IT4613)

ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

HỆ GỢI Ý

LỌC DỰA TRÊN PHIÊN

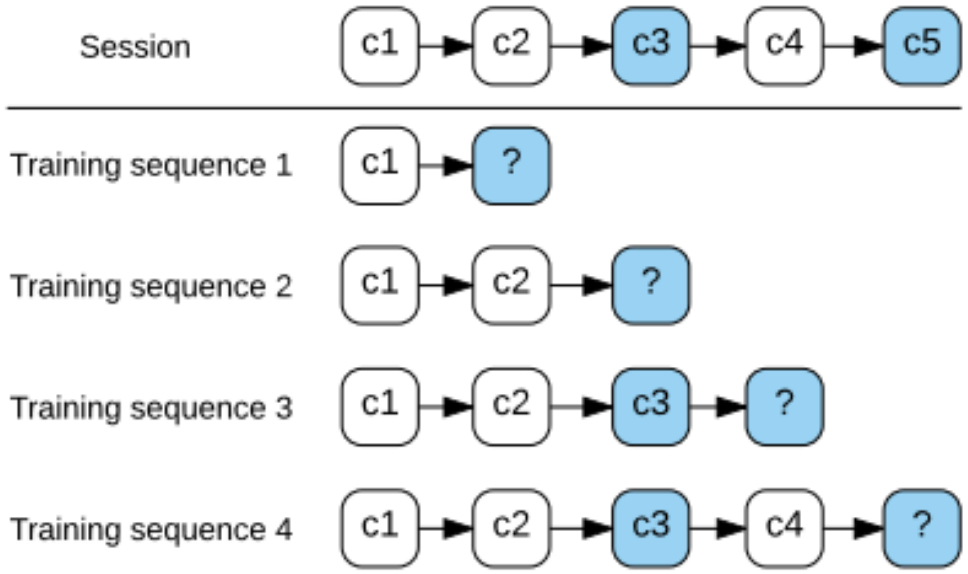
ONE LOVE. ONE FUTURE.

Nội dung môn học

- Giới thiệu về Hệ gợi ý
- Gợi ý dựa trên lọc cộng tác: Cơ bản và Các kỹ thuật nâng cao
- Gợi ý dựa trên nội dung
- Gợi ý dựa trên tri thức
- **Gợi ý dựa trên phiên**
- Kết hợp các kỹ thuật gợi ý
- Đánh giá hiệu quả đối với hệ gợi ý
- Ứng dụng kỹ thuật học sâu trong hệ gợi ý: Bài toán gợi ý tin tức
- Ứng dụng kỹ thuật học sâu trong hệ gợi ý: Học tương phản

Gợi ý dựa trên phiên (session-based recommendation system)

- Phiên làm việc của một user:



Gợi ý dựa trên phiên

- Khi nào sử dụng gợi ý dựa trên phiên
  - Khi hệ thống khó định danh user, chỉ có thông tin tương tác trong một phiên làm việc
  - Trong các hệ thống ưu tiên quan tâm đến các sở thích ngắn hạn
- Các cách tiếp cận
  - Tính độ tương đồng (item-to-item similarity, Item-Based *top-N* Recommendation)
  - Dựa trên tập mục thường xuyên (association rules)
  - Dựa trên phán đoán item tiếp theo trong chuỗi item trong phiên: Sử dụng mạng nơ-ron hồi quy (RNN)

1. Tính độ tương đồng (item-to-item)

- Biểu diễn vector item (thường dựa trên content, features)
- Với mỗi item, tính top K items tương đồng dựa trên độ đo tương đồng (e.g cosine similarity)
- Từ tập các items mà người dùng đã tương tác trong phiên, xây dựng tập các items ứng viên:
  - Tập con K items ứng viên: Mỗi item đã tương tác, lấy top K items tương đồng với item đã tương tác
  - Tập ứng viên là hợp của các tập con
  - Loại bỏ các items trùng lặp
- Xếp hạng các items ứng viên:
  - Tính tổng độ đo tương đồng của mỗi item ứng viên và các items mà user đã tương tác trong phiên
  - Sắp xếp và lấy top các items có tổng độ tương đồng cao nhất

Tính độ đo tương đồng

- Sử dụng độ đo tương đồng cosin
- Sử dụng xác suất có điều kiện:

$$P(u|v) = \frac{freq(uv)}{freq(v)}$$

Trong đó freq(uv) là số lượng users tương tác v sau đó tương tác u, freq(v) là số users tương tác v

$$Sim(u,v) = \frac{freq(uv)}{\big(freq(u)\big)^{\alpha} freq(v)}$$

α là hệ số điều chỉnh để phạt mức độ phổ biến của item u

2. Phát hiện các luật kết hợp

- Bài toán phát hiện luật kết hợp (Association rule mining)
  - Với một tập các giao dịch (transactions) cho trước, cần tìm các luật dự đoán khả năng xuất hiện trong một giao dịch của các mục (items) này dựa trên việc xuất hiện của các mục khác

TID	*Items*
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Coke
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Coke

Các ví dụ của luật kết hợp:

{Diaper} → {Beer}

{Milk, Bread} → {Eggs, Coke}

{Beer, Bread} → {Milk}

Các định nghĩa cơ bản (1)

- Tập mục (Itemset)
  - Một tập hợp gồm một hoặc nhiều mục
    - Ví dụ: {Milk, Bread, Diaper}
  - Tập mục mức k (k-itemset)
    - Một tập mục gồm k mục
- Tổng số hỗ trợ (Support count) σ
  - Số lần xuất hiện của một tập mục
  - Ví dụ: σ({Milk, Bread, Diaper}) = 2
- Độ hỗ trợ (Support) s
  - Tỷ lệ các giao dịch chứa một tập mục
  - Ví dụ: s({Milk, Bread, Diaper}) = 2/5
- Tập mục thường xuyên (Frequent/large itemset)
  - Một tập mục mà độ hỗ trợ lớn hơn hoặc bằng một giá trị ngưỡng minsup

TID	*Items*
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Coke
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Coke

Các định nghĩa cơ bản (2)

- Luật kết hợp (Association rule)
  - Một biểu thức kéo theo có dạng: $X \rightarrow Y$, trong đó X và Y là các tập mục
  - Ví dụ: {Milk, Diaper} → {Beer}
- Các độ đo đánh giá luật
  - Độ hỗ trợ (Support) s
    - Tỷ lệ các giao dịch chứa cả X và Y đối với tất cả các giao dịch
  - Độ tin cậy (Confidence) c
    - Tỷ lệ các giao dịch chứa cả X và Y đối với các giao dịch chứa X

TID	*Items*
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Coke
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Coke

$$\{\text{Milk}, \text{Diaper}\} \rightarrow \text{Beer}$$

$$s = \frac{\sigma(\text{Milk}, \text{Diaper}, \text{Beer})}{|\text{T}|} = \frac{2}{5} = 0.4$$

$$c = \frac{\sigma(\text{Milk}, \text{Diaper}, \text{Beer})}{\sigma(\text{Milk}, \text{Diaper})} = \frac{2}{3} = 0.67$$

Phát hiện các luật kết hợp

- Với một tập các giao dịch T, mục đích của bài toán phát hiện luật kết hợp là tìm ra tất cả các luật có:
  - độ hỗ trợ ≥ giá trị ngưỡng minsup, và
  - độ tin cậy ≥ giá trị ngưỡng minconf
- Cách tiếp cận vét cạn (Brute-force)
  - Liệt kê tất cả các luật kết hợp có thể
  - Tính toán độ hỗ trợ và độ tin cậy cho mỗi luật
  - Loại bỏ đi các luật có độ hỗ trợ nhỏ hơn minsup hoặc có độ tin cậy nhỏ hơn minconf

  $\Rightarrow$ Phương pháp vét cạn này có chi phí tính toán quá lớn, không áp dụng được trong thực tế!

Phát hiện luật kết hợp

- Tất cả các luật trên đều là sự phân tách (thành 2 tập con) của cùng tập mục : {Milk, Diaper, Beer}
- Các luật sinh ra từ cùng một tập mục sẽ có cùng độ hỗ trợ, nhưng có thể khác về độ tin cậy
- Do đó, trong quá trình phát hiện luật kết hợp, chúng ta có thể tách riêng 2 yêu cầu về độ hỗ trợ và độ tin cậy

TID	*Items*
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Coke
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Coke

Các luật kết hợp:

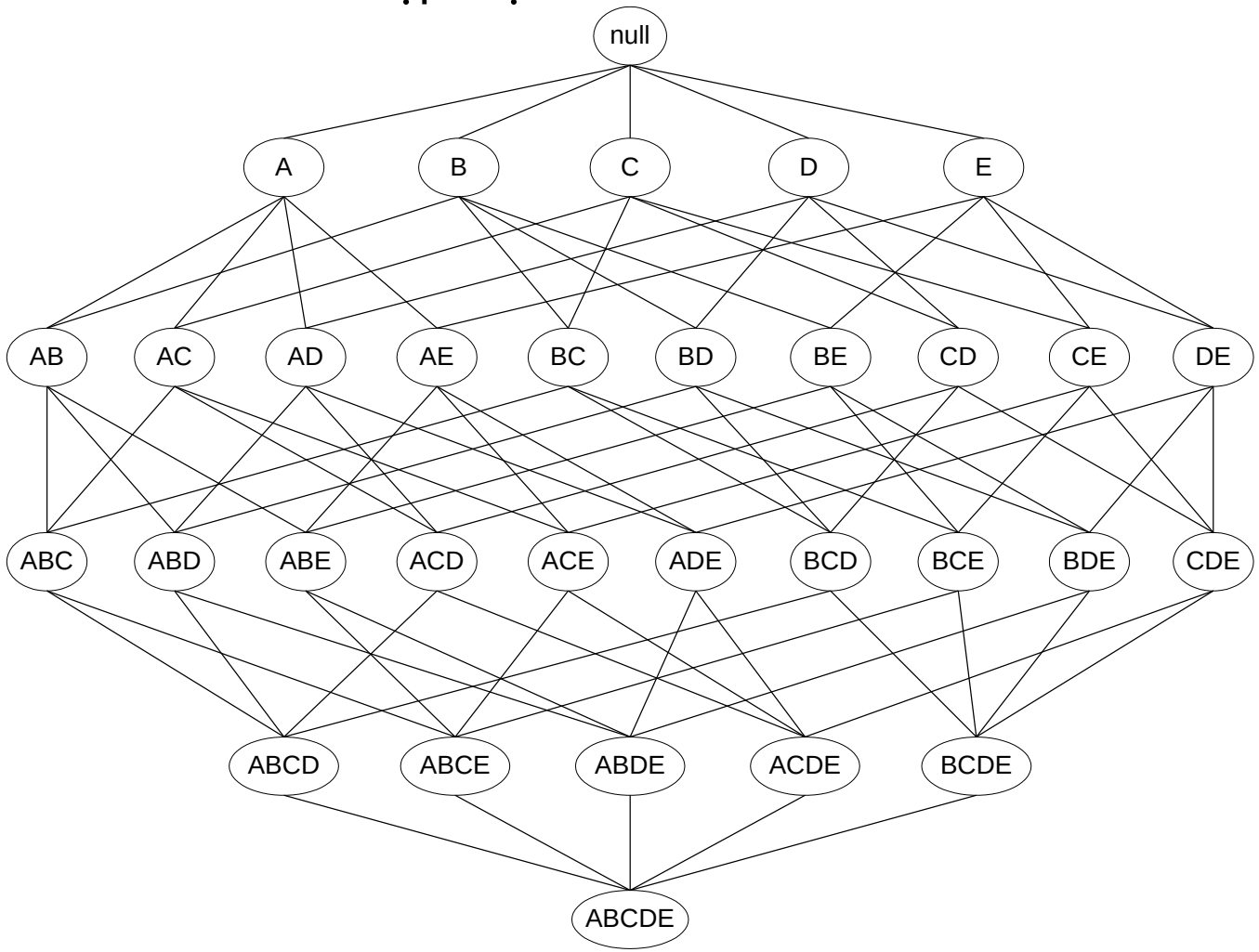
{Milk, Diaper} → {Beer} (s=0.4, c=0.67)
{Milk, Beer} → {Diaper} (s=0.4, c=1.0)
{Diaper, Beer} → {Milk} (s=0.4, c=0.67)
{Beer} → {Milk, Diaper} (s=0.4, c=0.67)
{Diaper} → {Milk, Beer} (s=0.4, c=0.5)
{Milk} → {Diaper, Beer} (s=0.4, c=0.5)

Phát hiện luật kết hợp

- Quá trình phát hiện luật kết hợp sẽ gồm 2 bước (2 giai đoạn) quan trọng:
  - Sinh ra các tập mục thường xuyên (frequent/large itemsets)
    - Sinh ra tất cả các tập mục có độ hỗ trợ ≥ minsup
  - Sinh ra các luật kết hợp
    - Từ mỗi tập mục thường xuyên (thu được ở bước trên), sinh ra tất cả các luật có độ tin cậy cao (≥ minconf)
    - Mỗi luật là một phân tách nhị phân (phân tách thành 2 phần) của một tập mục thường xuyên
- Bước sinh ra các tập mục thường xuyên (bước thứ 1) vẫn có chi phí tính toán quá cao!

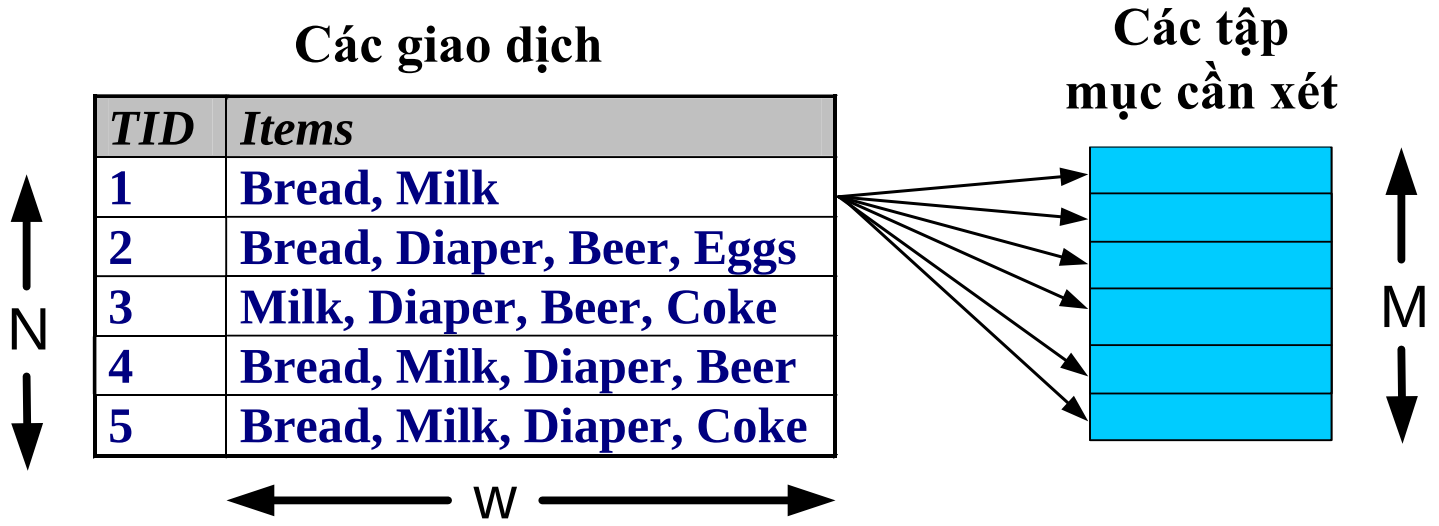
Lattice biểu diễn các tập mục cần xét

Với d mục, thì phải xét đến 2d các tập mục có thể!



Sinh ra các tập mục thường xuyên

- Phương pháp vét cạn (Brute-force)
  - Mỗi tập mục trong lattice đều được xét
  - Tính độ hỗ trợ của mỗi tập mục, bằng cách duyệt qua tất cả các giao dịch
  - Với mỗi giao dịch, so sánh nó với mỗi tập mục được xét
  - Độ phức tạp ~ O(N.M.w)
    - Với M = 2d, thì độ phức tạp này là quá lớn!



Các chiến lược sinh tập mục thường xuyên

- Giảm bớt số lượng các tập mục cần xét (M)
  - Tìm kiếm (xét) đầy đủ: M=2d
  - Sử dụng các kỹ thuật cắt tỉa (pruning) để giảm giá trị M
- Giảm bớt số lượng các giao dịch cần xét (N)
  - Giảm giá trị N, khi kích thước (số lượng các mục) của tập mục tăng lên
- Giảm bớt số lượng các so sánh (matchings/comparisons) giữa các tập mục và các giao dịch (N.M)
  - Sử dụng các cấu trúc dữ liệu phù hợp (hiệu quả) để lưu các tập mục cần xét hoặc các giao dịch
  - Không cần phải so sánh mỗi tập mục với mỗi giao dịch

Giảm bớt số lượng các tập mục cần xét

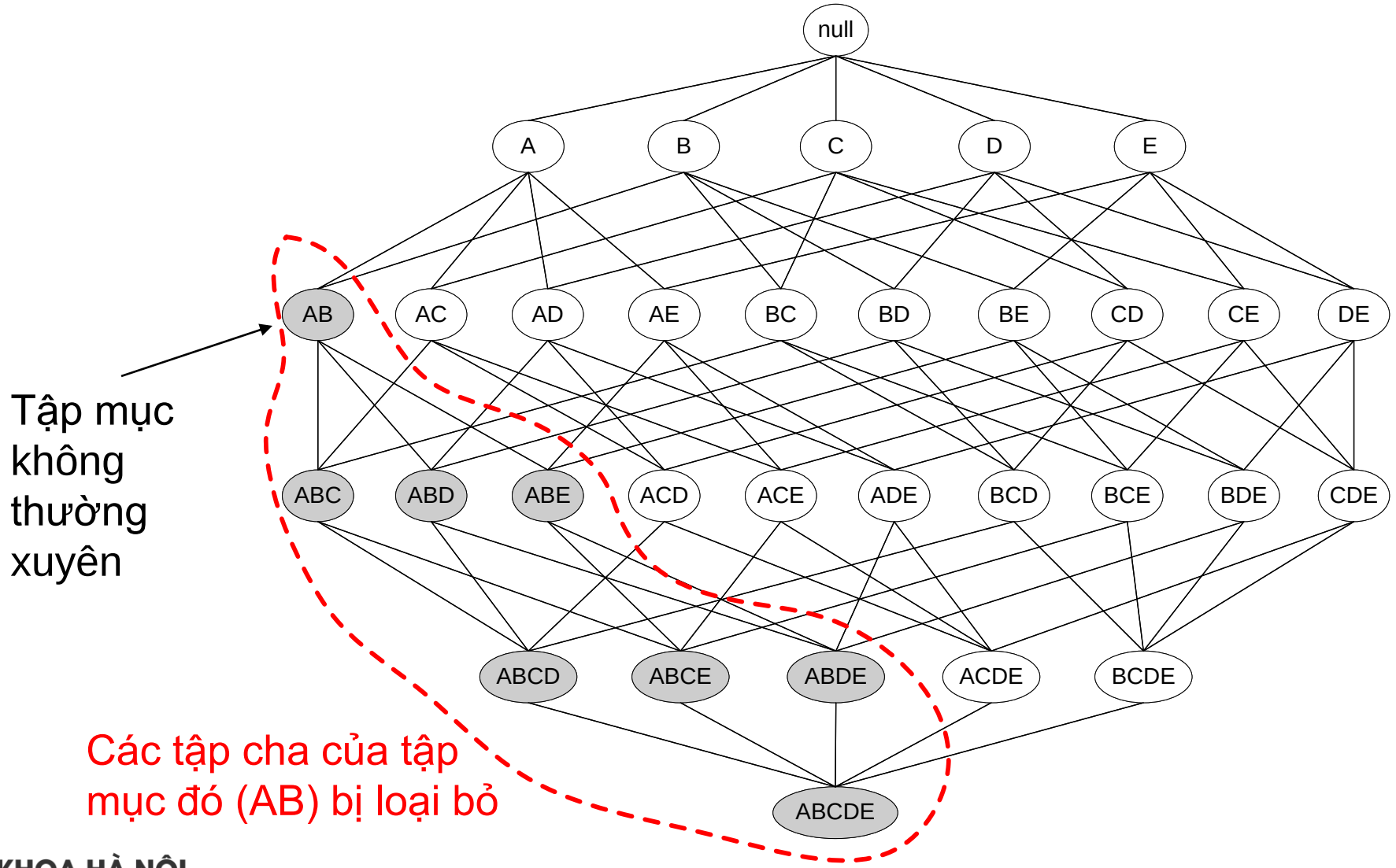
- Nguyên tắc của giải thuật Apriori – Loại bỏ (prunning) dựa trên độ hỗ trợ
  - Nếu một tập mục là thường xuyên, thì tất cả các tập con (subsets) của nó đều là các tập mục thường xuyên
  - Nếu một tập mục là không thường xuyên (not frequent), thì tất cả các tập cha (supersets) của nó đều là các tập mục không thường xuyên

- Nguyên tắc của giải thuật Apriori dựa trên đặc tính không đơn điệu (anti-monotone) của độ hỗ trợ

  - Độ hỗ trợ của một tập mục nhỏ hơn độ hỗ trợ của các tập con của nó

$$\forall X, Y : (X \subseteq Y) \Rightarrow s(X) \geq s(Y)$$

Apriori: Loại bỏ dựa trên độ hỗ trợ



Apriori: Loại bỏ dựa trên độ hỗ trợ

Item	Count
Bread	4
Coke	2
Milk	4
Beer	3
Diaper	4
Eggs	1

Các tập mục mức 1 (1-itemsets)

Itemset	Count
{Bread,Milk}	3
{Bread,Beer}	2
{Bread,Diaper}	3
{Milk,Beer}	2
{Milk,Diaper}	3
{Beer,Diaper}	3

Các tập mục mức 2 (2-itemsets)

(Không cần xét các tập mục có chứa mục *Coke* hoặc *Eggs*)

minsup = 3

- Nếu xét tất cả các tập mục có thể:
  $^6C_1 + {}^6C_2 + {}^6C_3 = 41$
- Với cơ chế loại bỏ dựa trên độ hỗ trợ:
  6 + 6 + 1 = 13

Các tập mục mức 3 (3-itemsets)

Itemset	Count
{Bread,Milk,Diaper}	3

Giải thuật Apriori

- Sinh ra tất cả các tập mục thường xuyên mức 1 (frequent 1-itemsets): các tập mục thường xuyên chỉ chứa 1 mục
- Gán k = 1
- Lặp lại, cho đến khi không có thêm bất kỳ tập mục thường xuyên nào mới
  - Từ các tập mục thường xuyên mức k (chứa k mục), sinh ra các tập mục mức (k+1) cần xét
  - Loại bỏ các tập mục mức (k+1) chứa các tập con là các tập mục không thường xuyên mức k
  - Tính độ hỗ trợ của mỗi tập mục mức (k+1), bằng cách duyệt qua tất cả các giao dịch
  - Loại bỏ các tập mục không thường xuyên mức (k+1)
  - Thu được các tập mục thường xuyên mức (k+1)

Sinh ra các luật kết hợp (1)

- Với mỗi tập mục thường xuyên L, cần tìm tất cả các tập con khác rỗng f $\subset$ L sao cho: f $\rightarrow$ {L \ f} thỏa mãn điều kiện về độ tin cậy tối thiểu
  - Vd: Với tập mục thường xuyên {A,B,C,D}, các luật cần xét gồm có:

$ABC \rightarrow D$, $ABD \rightarrow C$, $ACD \rightarrow B$, $BCD \rightarrow A$,
$A \rightarrow BCD$, $B \rightarrow ACD$, $C \rightarrow ABD$, $D \rightarrow ABC$
$AB \rightarrow CD$, $AC \rightarrow BD$, $AD \rightarrow BC$, $BC \rightarrow AD$,
$BD \rightarrow AC$, $CD \rightarrow AB$,

- Nếu |L| = k, thì sẽ phải xét (2k – 2) các luật kết hợp có thể (bỏ qua 2 luật: $L \rightarrow \varnothing$ và $\varnothing \rightarrow L$)

Sinh ra các luật kết hợp (2)

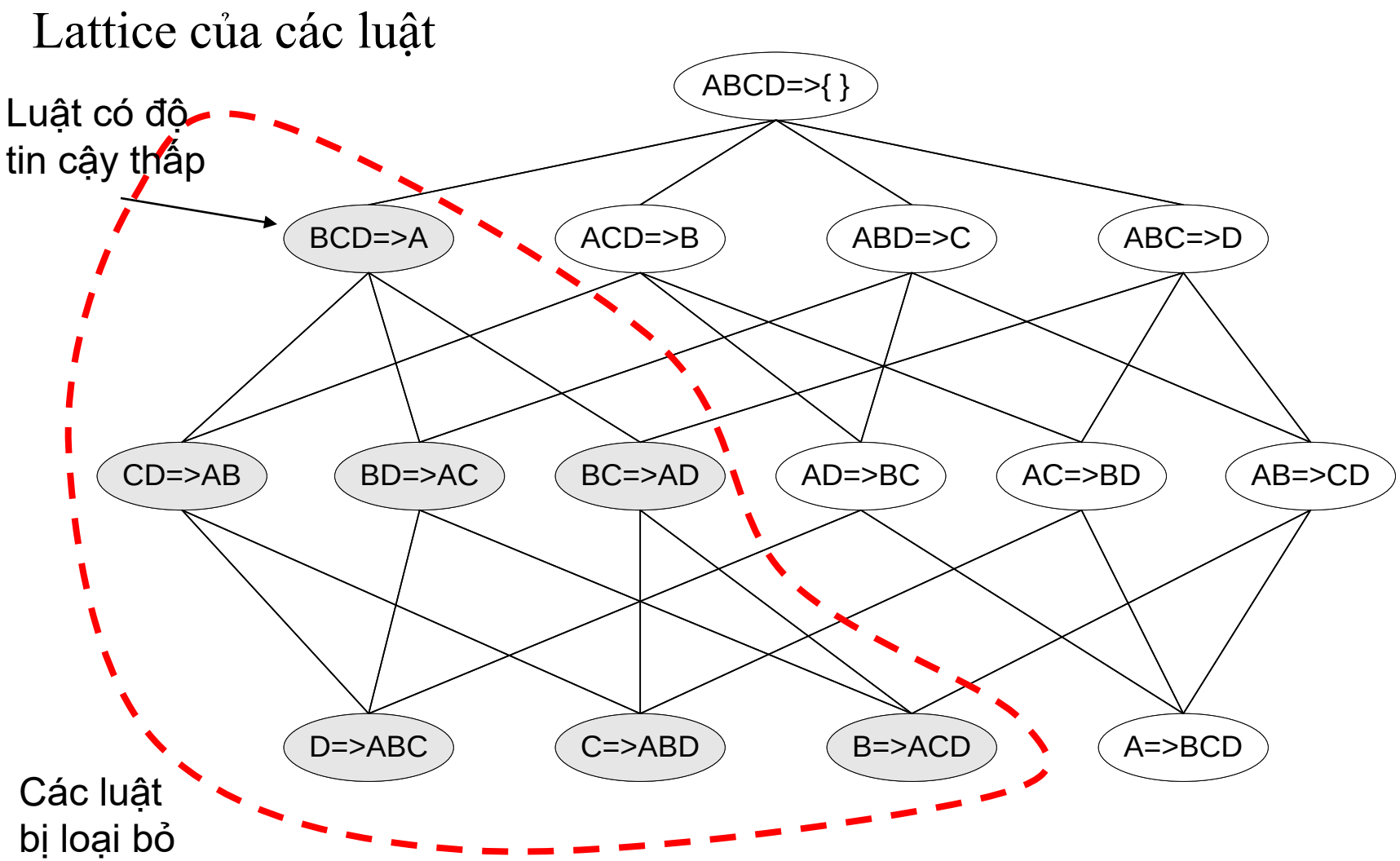
- Làm thế nào để sinh ra các luật từ các tập mục thường xuyên, một cách có hiệu quả?
- Xét tổng quát, độ tin cậy không có đặc tính không đơn điệu (anti-monotone)

  $c(ABC \rightarrow D)$ có thể lớn hơn hoặc nhỏ hơn $c(AB \rightarrow D)$

- Nhưng, độ tin cậy của các luật được sinh ra từ cùng một tập mục thường xuyên thì lại có đặc tính không đơn điệu
  - Ví dụ: Với L = {A,B,C,D}:

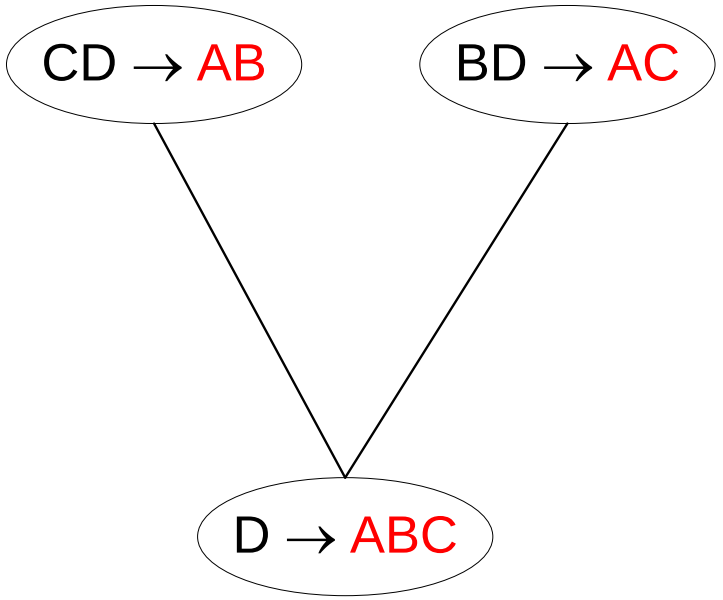
    $c(ABC \rightarrow D) \geq c(AB \rightarrow CD) \geq c(A \rightarrow BCD)$
  - Độ tin cậy có đặc tính không đơn điệu đối với số lượng các mục ở vế phải của luật

Apriori: Sinh ra các luật (1)



Apriori: Sinh ra các luật (2)

- Các luật cần xét được sinh ra bằng cách kết hợp 2 luật có cùng tiền tố (phần bắt đầu) của phần kết luận
(rule consequent)



- Ví dụ: Kết hợp 2 luật

  (CD → AB, BD → AC)
  sẽ sinh ra luật cần xét
  D → ABC

- Loại bỏ luật D → ABC nếu bất kỳ một luật con của nó (AD → BC, BCD → A, ...) không có độ tin cậy cao (< minconf)

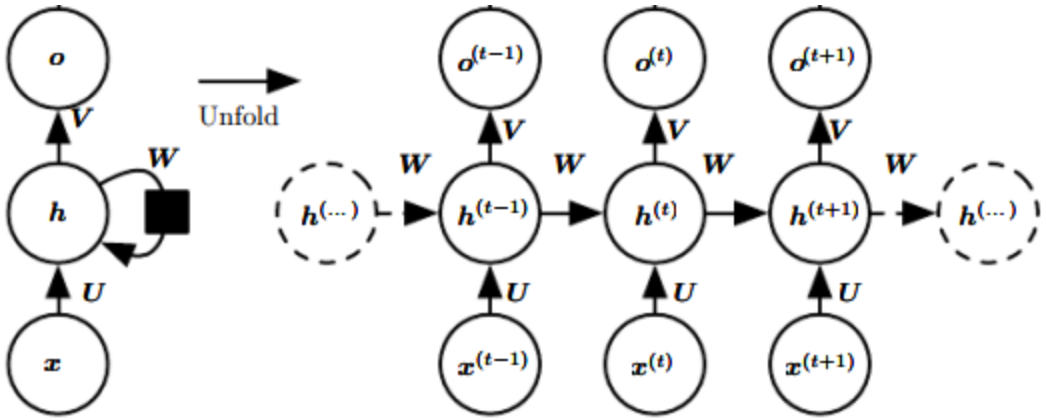
3. Tiếp cận dựa trên RNN

- Recurrent Neural Networks chuyên dùng cho dữ liệu dạng chuỗi.
- Ví dụ: Chuỗi: x1 -> x2 -> x3 -> x4 -> x5 -> x6
- Mỗi phần tử của chuỗi nằm ở một bước thời gian (time step) khác nhau và có quan hệ về mặt thứ tự với nhau
- Mô hình hóa quan hệ thứ tự giữa các phần tử trong chuỗi

=> Có thể áp dụng cho hệ gợi ý với chuỗi các items trong session

Mạng hồi quy (Recurrent Neural Networks)

❑ Cấu trúc chung của RNNs:



❑ x^t: item thứ t, o^t: item dự đoán tiếp theo

$$h^t = F(Wh^{t-1} + Ux^t)$$
$$o^t = G(Vh^t)$$

Mạng hồi quy (Recurrent Neural Networks)

- Sử dụng mạng RNN để gợi ý item tiếp theo
- Hàm Loss tại t dựa trên BPR:
- $L_t = -\frac{1}{N_s}\sum_{j=1}^{N_s} Log(sigmoid(o_{t,i} - o_{t,j}))$

Trong đó: i là item được tương tác tiếp theo, Ns là số negative samples j

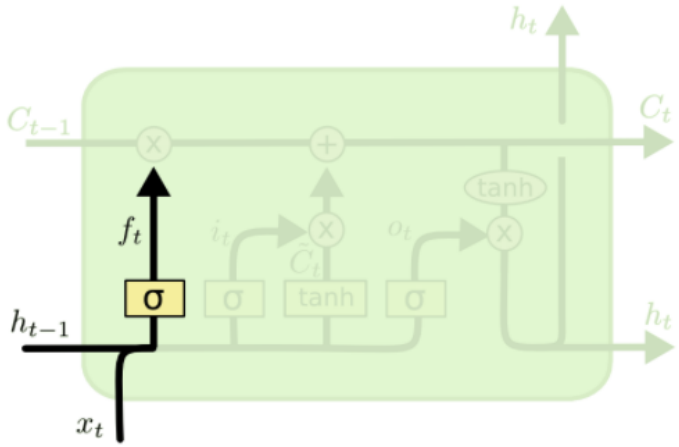
- RNN gặp vấn đề khi làm việc với chuỗi dài:
  - Do vanishing hoặc exploding gradient
- LTSM và GRU giúp giảm bớt ảnh hưởng này

Long Short-Term Memory

- Bao gồm các cổng để điều khiển luồng thông tin đi qua qua LSTM
- Thông tin được lưu trữ trong Memory cell (MC) Ct
- LSTM cell gồm 3 cổng:
    - > Forget gate: “quên” thông-tin-trong-MC-từ-timestep-trước, Ct-1
    - > Input gate: “lưu trữ” vào MC thông-tin-từ-timestep-hiện-tại, $\tilde{C}_t$
    - > Output gate: “trích xuất” thông tin từ MC để đưa ra ht

Long Short-Term Memory

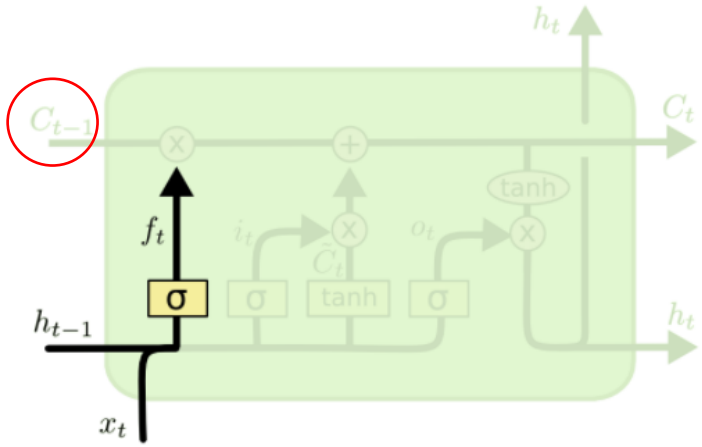
- Forget gate:
  - > thông-tin-trong-MC-từ-timestep-trước Ct-1
  - > “quên” Ct-1 nhiều hay ít ?



$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

Long Short-Term Memory

- Forget gate:
  - > thông-tin-trong-MC-từ-timestep-trước Ct-1
  - > “quên” Ct-1 nhiều hay ít ?

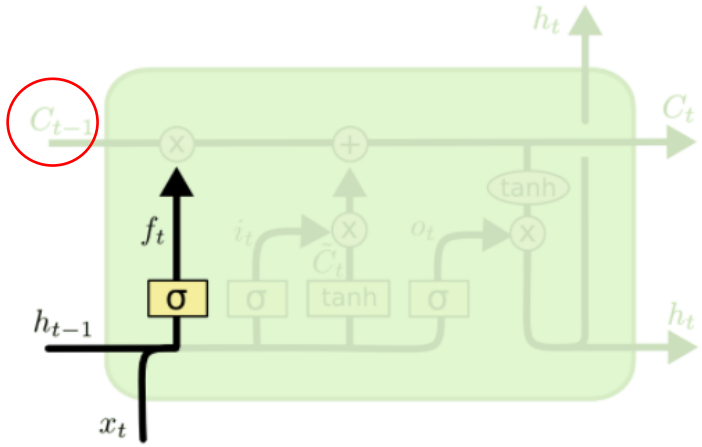


$f_t \in (0,1)$

$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

Long Short-Term Memory

- Forget gate:
  - > thông-tin-trong-MC-từ-timestep-trước Ct-1
  - > "quên" Ct-1 nhiều hay ít ?



$f_t \in (0,1)$

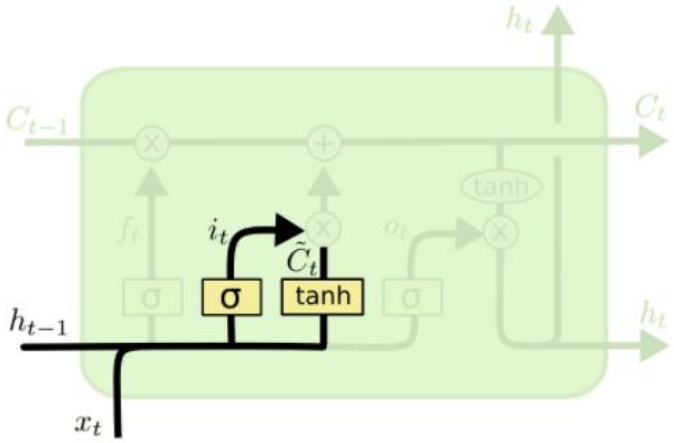
$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

Thông tin đi qua forget gate:

$f_t \; * C_{t-1}$

Long Short-Term Memory

- Input gate:
  - > dữ liệu từ timestep hiện tại xt
  - > thông-tin-từ-xt-muốn-cho-vào-MC là $\tilde{C}_t$, lấy bao nhiêu ?

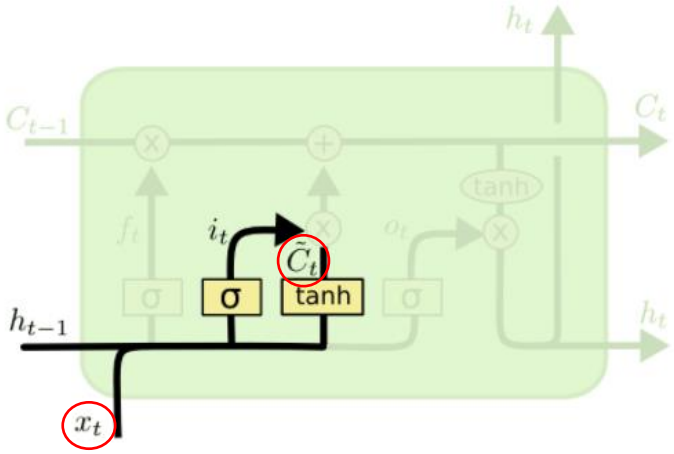


$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

Long Short-Term Memory

- Input gate:
  - > dữ liệu từ timestep hiện tại xt
  - > thông-tin-từ-xt-muốn-cho-vào-MC là $\tilde{C}_t$, lấy bao nhiêu ?

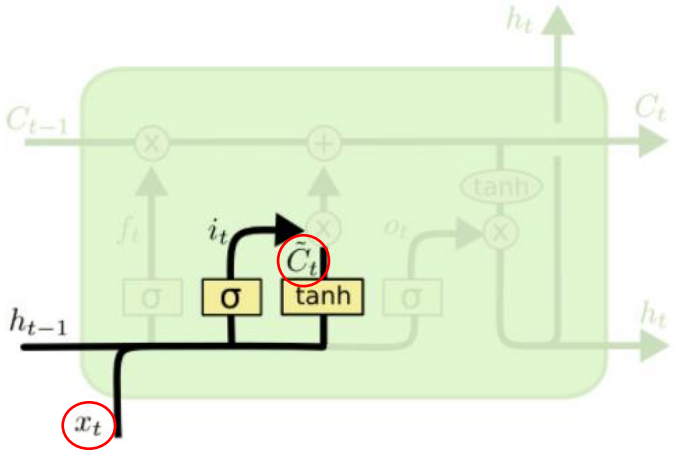


$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

Long Short-Term Memory

- Input gate:
  - > dữ liệu từ timestep hiện tại xt
  - > thông-tin-từ-xt-muốn-cho-vào-MC là $\tilde{C}_t$, lấy bao nhiêu ?



$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

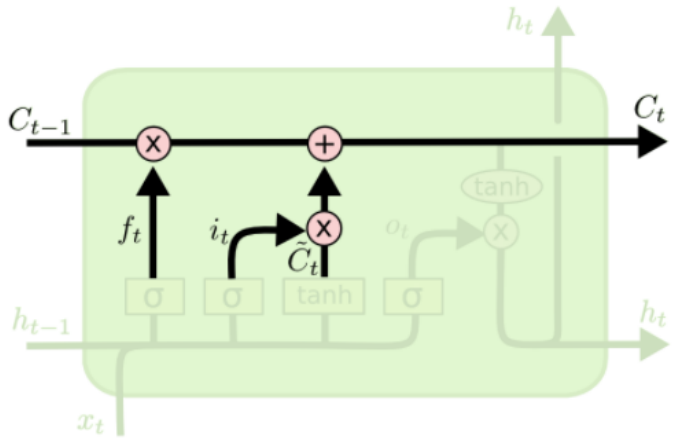
Thông tin đi qua input gate:

$$i_t \; * \tilde{C}_t$$

Long Short-Term Memory

❑ Tổng hợp thông tin:

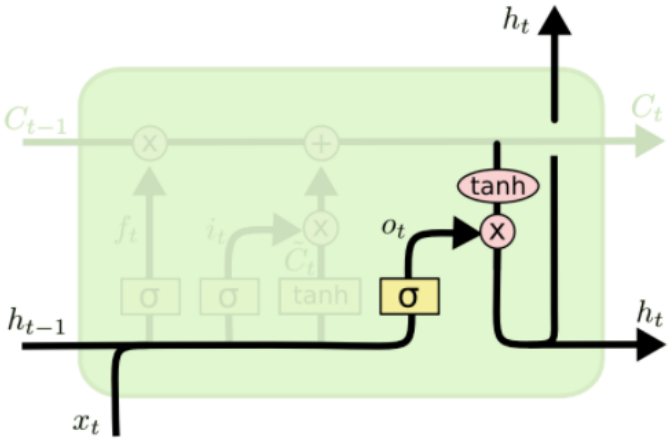
- > Thông tin đi qua forget gate: $f_t * C_{t-1}$
- > Thông tin đi qua input gate: $i_t * \widetilde{C}_t$



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Long Short-Term Memory

- Output gate:
  - > “trích xuất” thông tin từ MC, ht
  - > thông tin được trích xuất trước đó, ht-1

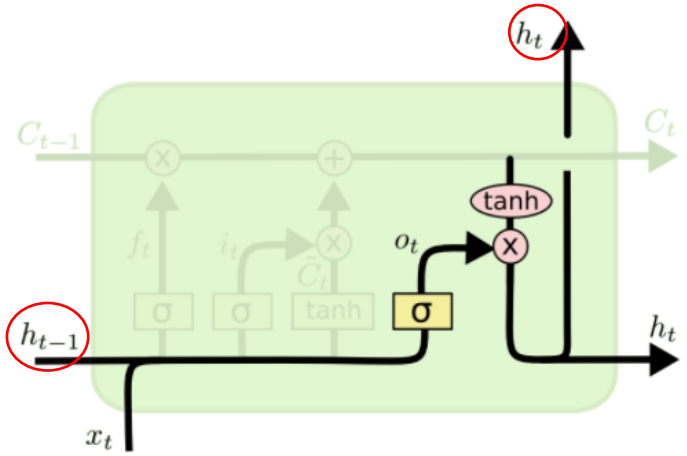


$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

Long Short-Term Memory

- Output gate:
  - > “trích xuất” thông tin từ MC, ht
  - > thông tin được trích xuất trước đó, ht-1



$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

Long Short-Term Memory

❑ Nhận xét:

> Memory cell có tác dụng là vật chứa thông tin nội bộ (Ct), được tích lũy qua các timesteps cho tới thời điểm t

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

> Output gate đưa ra ht , điều khiển việc trích xuất thông tin từ Memory cell

$$h_t = o_t * \tanh(C_t)$$

Long Short-Term Memory

❑ Nhận xét:

> Memory cell có tác dụng là vật chứa thông tin nội bộ (Ct),
được tích lũy qua các timesteps cho tới thời điểm t

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

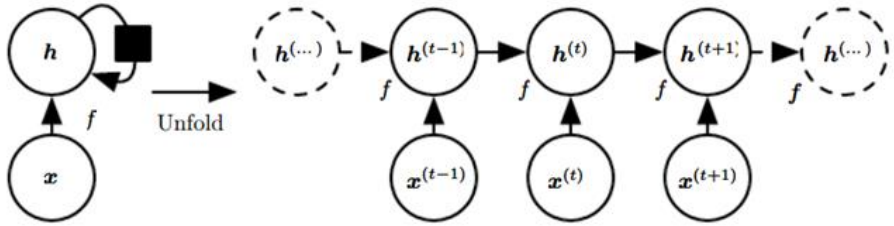
> Output gate đưa ra ht , điều khiển việc trích xuất
thông tin từ Memory cell

$$h_t = o_t * \tanh(C_t)$$

> C và h là 2 vector có cùng kích thước -> LSTM cell size (lstm size)

Long Short-Term Memory

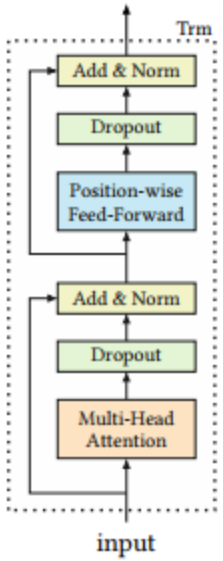
- Cách sử dụng LSTM:



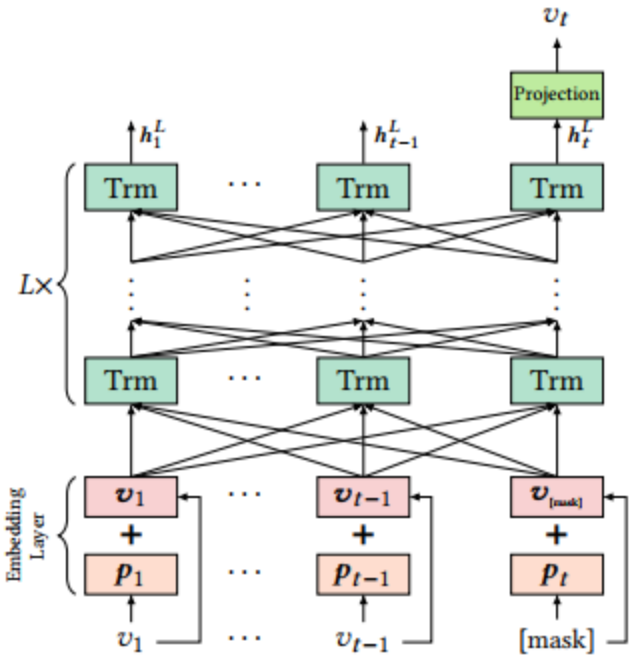
> cần có trạng thái khởi đầu h(0)

> trong LSTM, memory cell cũng cần có trạng thái khởi C(0)

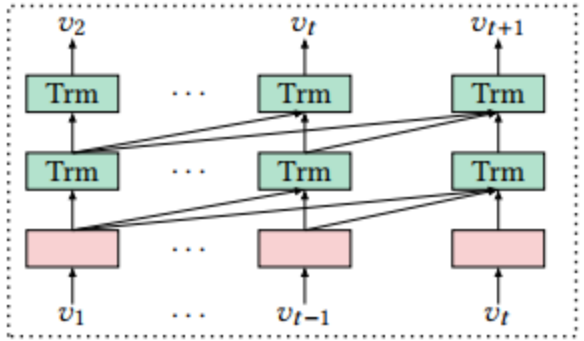
Một số mở rộng: SaSRec và Bert4rec



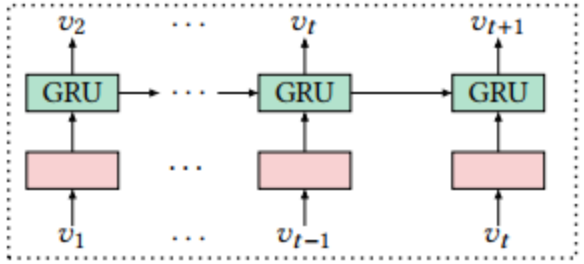
(a) Transformer Layer.



(b) BERT4Rec model architecture.



(c) SASRec model architecture.



(d) RNN based sequential recommendation methods.

Kết luận

- Gợi ý dựa trên phiên được sử dụng khi khó định danh user và quan tâm đến các sở thích ngắn hạn
- 3 cách tiếp cận phổ biến:
  - Dựa trên độ đo tương đồng item-to-item trong phiên
  - Dựa trên tập mục thường xuyên
  - Dựa trên mạng RNN

HUST

hust.edu.vn fb.com/dhbkhn

THANK YOU !